

Factory Method Design Pattern — Bilingual Study Document (English + Hindi)

Inspired by refactoring.guru/design-patterns/factory-method — concepts and structure preserved, written in original wording with self-authored PHP examples

Study Companion · Updated July 2026

Factory Method Design Pattern

Note on sourcing / स्रोत पर टिप्पणी: refactoring.guru is commercially operated content — the site sells an eBook and a course, and its written explanations, diagrams, and code samples are copyrighted. This document follows the exact section order of their Factory Method page (Intent → Problem → Solution → Structure → Pseudocode → Applicability → How to Implement → Pros and Cons → Relations with Other Patterns → Code Examples), because that structure is a genuinely useful teaching sequence — but every sentence below is written independently in my own words, and all PHP code is original, not copied from their site.

refactoring.guru व्यावसायिक रूप से संचालित (commercially operated) कंटेंट है — यह साइट एक eBook और एक कोर्स बेचती है, और इसकी लिखित व्याख्याएँ, डायग्राम्स, और कोड सैंपल्स कॉपीराइट हैं। यह दस्तावेज़ उनके Factory Method पेज के ठीक उसी सेक्शन-क्रम को फ़ॉलो करता है (Intent → Problem → Solution → Structure → Pseudocode → Applicability → How to Implement → Pros and Cons → Relations with Other Patterns → Code Examples), क्योंकि वह क्रम वाक़ई एक उपयोगी टीचिंग सीक्वेंस है — लेकिन नीचे का हर वाक्य मेरे अपने शब्दों में स्वतंत्र रूप से लिखा गया है, और सारा PHP कोड ओरिजिनल है, उनकी साइट से कॉपी नहीं किया गया।

Intent

Factory Method is a creational design pattern. It gives a superclass an interface for creating objects, while letting each subclass decide exactly which concrete class actually gets instantiated.

Factory Method एक क्रिएशनल (creational) डिज़ाइन पैटर्न है। यह एक सुपरक्लास को ऑब्जेक्ट्स बनाने के लिए एक इंटरफ़ेस देता है, जबकि हर सबक्लास को यह तय करने देता है कि असल में कौन-सी कॉन्क्रीट क्लास इंस्टैंशिएट होगी।

Also known by another name worth knowing for interviews: the **Virtual Constructor**, since the factory method behaves like a constructor whose exact behavior is overridden per subclass.

इंटरव्यू के लिए जानने लायक एक और नाम: **वर्चुअल कंस्ट्रक्टर (Virtual Constructor)**, क्योंकि factory मेथड एक ऐसे कंस्ट्रक्टर की तरह व्यवहार करता है जिसका ठीक-ठीक व्यवहार हर सबक्लास में अलग तरह से ओवरराइड किया जाता है।

Problem

Consider a logistics-management application being built from scratch. In its first version, the only supported mode of transport is trucks, so nearly all the application's logic sits inside a single `Truck` class.

एक लॉजिस्टिक्स-मैनेजमेंट ऐप्लिकेशन की कल्पना कीजिए, जो शुरू से बनाया जा रहा है। इसके पहले वर्शन में, सिर्फ ट्रकों से ट्रांसपोर्ट सपोर्टेड है, इसलिए ऐप्लिकेशन का लगभग सारा लॉजिक एक ही `Truck` क्लास के अंदर है।

The application does well, and soon enough sea-transport companies start requesting that ships be supported too. Reasonable enough — except almost every part of the existing code is already tightly wired to the `Truck` class specifically. Introducing `Ship` cleanly would require touching a large fraction of that code, and if a third mode of transport (rail, say) shows up next quarter, the same disruptive rewrite repeats.

ऐप्लिकेशन अच्छा चलता है, और जल्द ही समुद्री-परिवहन (sea-transport) कंपनियाँ जहाज़ों (ships) को भी सपोर्ट करने की माँग करने लगती हैं। यह माँग वाजिब है — सिवाय इसके कि मौजूदा कोड का लगभग हर हिस्सा पहले से खास तौर पर `Truck` क्लास से कसकर जुड़ा हुआ है। `Ship` को सही तरीके से जोड़ने के लिए उस कोड के एक बड़े हिस्से को छूना पड़ेगा, और अगर अगली तिमाही में परिवहन का तीसरा तरीका (मान लीजिए रेल) आ जाए, तो वही उथल-पुथल भरा फिर से लिखना दोहराया जाएगा।

Left unaddressed, this pattern of growth produces code stuffed with conditional branches that switch behavior depending on which transport class is in play — precisely the shape of code that becomes harder to extend with every addition, not easier.

अगर इसे ठीक न किया जाए, तो बढ़ोतरी का यह पैटर्न ऐसा कोड बना देता है जो शर्तों वाली शाखाओं (conditional branches) से भरा हो — जो इस आधार पर व्यवहार बदलती हैं कि कौन-सी ट्रांसपोर्ट क्लास इस्तेमाल हो रही है — बिल्कुल वही बनावट, जो हर नई चीज़ जोड़ने के साथ आसान होने के बजाय कठिन होती जाती है।

Solution

The Factory Method pattern's suggestion: stop constructing objects directly with `new` scattered throughout the business logic, and route that construction through one dedicated *factory method* instead. The objects the factory method hands back are usually called **products**.

Factory Method पैटर्न का सुझाव: बिज़नेस लॉजिक में जगह-जगह बिखरे `new` से सीधे ऑब्जेक्ट्स बनाना बंद कीजिए, और उस निर्माण (construction) को एक समर्पित (dedicated) *factory* मेथड से गुज़ारिए। *factory* मेथड जो ऑब्जेक्ट्स वापस देता है, उन्हें आमतौर पर **प्रोडक्ट्स (products)** कहा जाता है।

At first this may look like nothing more than moving a constructor call from one place to another. The real payoff shows up once you notice that a factory method can be *overridden* — a subclass can override it and hand back a different concrete product type entirely, while the calling code keeps working unchanged.

पहली नज़र में यह सिर्फ़ एक कंस्ट्रक्टर कॉल को एक जगह से दूसरी जगह ले जाने जैसा लग सकता है। असली फ़ायदा तब दिखता है जब आप नोटिस करते हैं कि *factory* मेथड को ओवरराइड किया जा सकता है — एक सबक्लास इसे ओवरराइड करके पूरी तरह अलग कॉन्क्रीट प्रोडक्ट टाइप वापस दे सकता है, जबकि कॉलिंग कोड बिना बदले काम करता रहता है।

There is one condition attached: every concrete product a subclass might return has to share a common interface or base class, and the factory method's declared return type in the base class must be that shared interface — not any one specific concrete type.

एक शर्त जुड़ी है: हर कॉन्क्रीट प्रोडक्ट, जो कोई सबक्लास लौटा सकता है, उसे एक साझा इंटरफ़ेस या बेस क्लास साझा करना ही होगा, और बेस क्लास में *factory* मेथड का डिक्लेयर किया गया रिटर्न टाइप वही साझा इंटरफ़ेस होना चाहिए — कोई एक खास कॉन्क्रीट टाइप नहीं।

Back to the logistics example: both `Truck` and `Ship` would implement a shared `Transport` interface declaring a `deliver` method — trucks deliver by road, ships deliver by sea, each in its own way, but both answer to the same method name. A `RoadLogistics` class's factory method returns trucks; a `SeaLogistics` class's factory method returns ships. The calling code — the *client* — never has to know or care which one it's holding; it only relies on the `Transport` interface's contract.

लॉजिस्टिक्स उदाहरण पर वापस: `Truck` और `Ship` दोनों एक साझा `Transport` इंटरफ़ेस इम्प्लीमेंट करेंगे, जो एक `deliver` मेथड डिक्लेयर करता है — ट्रक सड़क से डिलीवर करते हैं, जहाज़ समुद्र से, हर एक अपने तरीक़े से, लेकिन दोनों एक ही मेथड नाम का जवाब देते हैं। एक `RoadLogistics` क्लास का *factory* मेथड ट्रक लौटाता है; एक `SeaLogistics` क्लास का *factory* मेथड जहाज़ लौटाता है। कॉलिंग कोड — क्लाइंट — को कभी यह जानने या परवाह करने की ज़रूरत नहीं पड़ती कि वह कौन-सा पकड़े हुए है; यह सिर्फ़ `Transport` इंटरफ़ेस के कॉन्ट्रैक्ट पर भरोसा करता है।

Structure

Four participants make up the pattern:

पैटर्न को चार भागीदार (participants) मिलकर बनाते हैं:

Product — declares the interface shared by everything the creator and its subclasses are capable of producing.

Product — वह इंटरफ़ेस डिक्लेयर करता है, जिसे creator और उसकी सबक्लासेज़ जो कुछ भी बना सकती हैं, वे सब साझा करते हैं।

Concrete Products — the different, specific implementations of that product interface.

Concrete Products — उस प्रोडक्ट इंटरफ़ेस के अलग-अलग, खास इम्प्लीमेंटेशन्स।

Creator — declares the factory method, whose return type must match the product interface. It can be declared `abstract`, forcing every subclass to supply its own implementation, or it can return some sensible default product. Worth flagging clearly: despite the name, creating products is not actually the Creator's central job — the Creator typically holds real business logic that operates on products, and the factory method exists to keep that business logic decoupled from any one concrete product class.

Creator — factory मेथड डिक्लेयर करता है, जिसका रिटर्न टाइप प्रोडक्ट इंटरफ़ेस से मेल खाना चाहिए। इसे `abstract` डिक्लेयर किया जा सकता है, जो हर सबक्लास को अपना खुद का इम्प्लीमेंटेशन देने पर मजबूर करता है, या यह कोई उचित डिफ़ॉल्ट प्रोडक्ट लौटा सकता है। साफ़ तौर पर बताने लायक बात: नाम के बावजूद, प्रोडक्ट्स बनाना असल में Creator का मुख्य काम नहीं है — Creator में आमतौर पर असली बिज़नेस लॉजिक होता है, जो प्रोडक्ट्स पर काम करता है, और factory मेथड इसीलिए है ताकि वह बिज़नेस लॉजिक किसी एक कॉन्क्रीट प्रोडक्ट क्लास से अलग रहे।

Concrete Creators — override the base factory method to return a different, specific product type. Note also that a factory method doesn't strictly need to construct a brand-new object every time — returning an existing instance from a cache or pool is equally valid.

Concrete Creators — बेस factory मेथड को ओवरराइड करके एक अलग, खास प्रोडक्ट टाइप लौटाते हैं। यह भी ध्यान देने लायक है कि factory मेथड को हर बार बिल्कुल नया ऑब्जेक्ट बनाना ज़रूरी नहीं — किसी कैश या पूल से मौजूदा इंस्टेंस लौटाना भी उतना ही मान्य (valid) है।

Pseudocode

The illustrative scenario here is a cross-platform UI toolkit, showing how Factory Method lets client code work with dialogs and buttons without being coupled to any operating-system-specific classes.

यहाँ का उदाहरण-परिदृश्य एक क्रॉस-प्लेटफ़ॉर्म UI टूलकिट है, जो दिखाता है कि Factory Method कैसे क्लाइंट कोड को डायलॉग्स और बटन्स के साथ काम करने देता है, बिना किसी ऑपरेटिंग-सिस्टम-विशिष्ट क्लास से जुड़े।

A base `Dialog` class renders its window using UI elements that look slightly different per operating system but must behave the same way everywhere — a button remains a button, on Windows or on the web.

एक बेस `Dialog` क्लास अपनी विंडो को UI एलिमेंट्स के ज़रिए रेंडर करती है, जो हर ऑपरेटिंग सिस्टम पर थोड़े अलग दिखते हैं लेकिन हर जगह एक जैसा व्यवहार करने चाहिए — एक बटन, Windows पर हो या वेब पर, बटन ही रहता है।

Rather than rewriting `Dialog`'s logic per platform, a factory method that produces buttons is declared on the base class; a `WindowsDialog` subclass overrides it to hand back Windows-styled buttons, inheriting everything else `Dialog` already does. For the base `Dialog` class to remain functional regardless of which button type it's handed, it must only ever work against an abstract `Button` interface — never a concrete button class directly. Push this idea to more UI elements at once, and it edges toward the related **Abstract Factory** pattern, covered separately.

`Dialog` के लॉजिक को हर प्लेटफ़ॉर्म के लिए दोबारा लिखने के बजाय, बेस क्लास पर एक factory मेथड डिक्लेयर किया जाता है जो बटन्स बनाता है; एक `WindowsDialog` सबक्लास इसे ओवरराइड करके Windows-स्टाइल बटन्स वापस देती है, बाक़ी सब कुछ `Dialog` से इनहेरिट करते हुए। बेस `Dialog` क्लास को काम करते रहने के लिए, चाहे उसे कोई भी बटन टाइप मिले, उसे हमेशा सिर्फ़ एक ऐब्स्ट्रैक्ट `Button` इंटरफ़ेस के खिलाफ़ काम करना चाहिए — कभी सीधे किसी कॉन्क्रीट बटन क्लास के खिलाफ़ नहीं। इस विचार को एक साथ कई UI एलिमेंट्स तक बढ़ाया जाए, तो यह संबंधित **Abstract Factory** पैटर्न के करीब पहुँचता है, जिसे अलग से कवर किया जाता है।

(Original PHP demonstrating this exact Dialog/Button scenario appears in the Implementation section below and in the standalone companion file.)

(इसी `Dialog/Button` परिदृश्य को दिखाने वाला ओरिजिनल PHP नीचे `Implementation` सेक्शन में और स्टैंडअलोन companion फ़ाइल में मिलेगा।)

Applicability

Reach for Factory Method when the exact types — and the dependencies — your code will need to work with aren't known in advance.

Factory Method का इस्तेमाल तब कीजिए जब वे ठीक-ठीक टाइप्स — और उनकी डिपेंडेंसीज़ — जिनके साथ आपके कोड को काम करना है, पहले से मालूम नहीं हैं।

The pattern separates product-construction code from product-usage code, so the construction side can be extended on its own — adding a new product typically only requires one new creator subclass, no edits to the code that already uses products.

पैटर्न, प्रोडक्ट बनाने वाले कोड को प्रोडक्ट इस्तेमाल करने वाले कोड से अलग करता है, ताकि निर्माण (construction) वाला हिस्सा अपने आप बढ़ाया जा सके — एक नया प्रोडक्ट जोड़ने के लिए आमतौर पर सिर्फ़ एक नई creator सबक्लास चाहिए, उस कोड में कोई बदलाव नहीं जो पहले से प्रोडक्ट्स इस्तेमाल कर रहा है।

Reach for it, too, when building a library or framework and wanting to let consumers extend its internal components. Inheritance alone extends behavior, but doesn't tell the framework which subclass to actually instantiate at runtime — collapsing the construction logic into one overridable factory method solves exactly that: extend a `Button` into a custom `RoundButton`, then override the framework's factory method in a subclass to return `RoundButton` instead of the default, and the rest of the framework keeps working unmodified.

इसका इस्तेमाल तब भी कीजिए जब कोई लाइब्रेरी या फ्रेमवर्क बना रहे हों और उपयोगकर्ताओं को इसके आंतरिक घटकों (internal components) को एक्सटेंड करने देना चाहते हों। सिर्फ़ इनहेरिटेंस व्यवहार को एक्सटेंड करता है, लेकिन फ्रेमवर्क को यह नहीं बताता कि रनटाइम पर असल में कौन-सी सबक्लास इंस्टैंशिएट करनी है — निर्माण लॉजिक को एक ओवरराइड-करने-योग्य factory मेथड में समेटना ठीक यही हल करता है: एक `Button` को एक कस्टम `RoundButton` में एक्सटेंड कीजिए, फिर फ्रेमवर्क के factory मेथड को एक सबक्लास में ओवरराइड कीजिए ताकि डिफ़ॉल्ट के बजाय `RoundButton` लौटे, और बाक़ी फ्रेमवर्क बिना बदले काम करता रहे।

Reach for it, finally, when the goal is reusing existing objects instead of rebuilding costly ones from scratch every time — database connections, file handles, and similar resource-heavy objects being the classic case. A plain constructor is defined to always return a fresh instance; it structurally cannot hand back an existing one. A factory method has no such restriction, so it's the natural home for pooling logic: check a pool for a free object first, hand it back if found, construct and register a new one only if not.

अंत में, इसका इस्तेमाल तब कीजिए जब लक्ष्य हर बार महँगे ऑब्जेक्ट्स को शुरू से फिर से बनाने के बजाय मौजूदा ऑब्जेक्ट्स का पुनः इस्तेमाल (reuse) करना हो — डेटाबेस कनेक्शन्स, फ़ाइल हैंडल्स, और ऐसे ही रिसोर्स-भारी ऑब्जेक्ट्स इसका क्लासिक उदाहरण हैं। एक साधारण कंस्ट्रक्टर हमेशा एक ताज़ा (fresh) इंस्टेंस लौटाने के लिए ही परिभाषित होता है; यह संरचनात्मक रूप से (structurally) मौजूदा इंस्टेंस वापस नहीं दे सकता। factory मेथड पर ऐसी कोई पाबंदी नहीं है, इसलिए यह पूलिंग लॉजिक के लिए स्वाभाविक जगह है: पहले किसी पूल में एक खाली ऑब्जेक्ट देखिए, मिले तो वापस दीजिए, न मिले तभी नया बनाइए और रजिस्टर कीजिए।

How to Implement

Step 1. Give every product a shared interface, declaring only the methods that genuinely make sense for all of them.

चरण 1. हर प्रोडक्ट को एक साझा इंटरफ़ेस दीजिए, जिसमें सिर्फ़ वे मेथड्स डिक्लेयर हों जो सचमुच सभी के लिए मायने रखते हैं।

Step 2. Add an empty factory method to the creator class, its return type set to that shared interface.

चरण 2. creator क्लास में एक खाली factory मेथड जोड़िए, जिसका रिटर्न टाइप उसी साझा इंटरफ़ेस पर सेट हो।

Step 3. Hunt down every place in the creator's code that constructs a product directly, and replace each with a call to the factory method — moving the actual construction logic into the factory method itself. This step may temporarily need a parameter to control which product gets built, and the factory method's body may look genuinely messy at this point (a sprawling `switch`, most likely) — that's expected and gets cleaned up in the next step.

चरण 3. creator के कोड में हर उस जगह को ढूँढ़िए जो सीधे एक प्रोडक्ट बनाती है, और हर एक को factory मेथड की कॉल से बदल दीजिए — असली निर्माण लॉजिक को factory मेथड के अंदर ले जाते हुए। इस चरण में अस्थायी रूप से एक पैरामीटर की ज़रूरत पड़ सकती है, यह नियंत्रित करने के लिए कि कौन-सा प्रोडक्ट बनना है, और इस बिंदु पर factory मेथड की बॉडी सचमुच अस्त-व्यस्त (एक फैला हुआ `switch`, संभवतः) दिख सकती है — यह अपेक्षित है और अगले चरण में साफ़ हो जाता है।

Step 4. Create one creator subclass per product type the factory method currently handles, override the factory method in each, and move the relevant slice of construction logic there.

चरण 4. factory मेथड फ़िलहाल जिन प्रोडक्ट टाइप्स को संभालता है, हर एक के लिए एक creator सबक्लास बनाइए, हर एक में factory मेथड को ओवरराइड कीजिए, और निर्माण लॉजिक का संबंधित हिस्सा वहाँ ले जाइए।

Step 5. If the product count is high enough that a subclass per product stops making sense, the base class's control parameter can be reused inside subclasses instead — for example, a `GroundMail` subclass working with both `Truck` and `Train` objects by accepting a parameter, rather than spawning a separate `TrainMail` subclass just for that one combination.

चरण 5. अगर प्रोडक्ट्स की संख्या इतनी ज़्यादा हो कि हर प्रोडक्ट के लिए एक सबक्लास बनाना अब समझदारी न लगे, तो बेस क्लास का कंट्रोल पैरामीटर सबक्लासेज़ के अंदर दोबारा इस्तेमाल किया जा सकता है — उदाहरण के लिए, एक `GroundMail` सबक्लास, `Truck` और `Train` दोनों ऑब्जेक्ट्स के साथ एक पैरामीटर स्वीकार करके काम करे, बजाय इसके कि सिर्फ़ इसी एक संयोजन के लिए एक अलग `TrainMail` सबक्लास बनाई जाए।

Step 6. If, after all this extraction, the base factory method's body ends up completely empty, mark it `abstract`. If some sensible default behavior remains, keep it as the default implementation.

चरण 6. अगर इस पूरे निष्कर्षण (extraction) के बाद, बेस factory मेथड की बॉडी पूरी तरह खाली रह जाती है, तो इसे `abstract` चिह्नित कीजिए। अगर कोई उचित डिफ़ॉल्ट व्यवहार बचता है, तो उसे डिफ़ॉल्ट इम्प्लीमेंटेशन के तौर पर रखिए।

Pros and Cons

Pros: tight coupling between the creator and its concrete products is avoided; product-creation code collapses into one place, aligning with the Single Responsibility Principle and making the codebase easier to maintain; new product types can be introduced without touching client code already relying on the existing ones, aligning with the Open/Closed Principle.

फ़ायदे: creator और उसके कॉन्क्रीट प्रोडक्ट्स के बीच कसा हुआ कपलिंग (tight coupling) टाला जाता है; प्रोडक्ट-निर्माण कोड एक ही जगह सिमट जाता है, जो Single Responsibility Principle के अनुरूप है और कोडबेस को बनाए रखना आसान बनाता है; नए प्रोडक्ट टाइप्स को उस क्लाइंट कोड को छुए बिना जोड़ा जा सकता है जो पहले से मौजूद प्रोडक्ट्स पर निर्भर है, जो Open/Closed Principle के अनुरूप है।

Cons: the code can end up more complex than before, since applying the pattern typically means introducing a batch of new subclasses. It goes down easiest when it's being introduced into a creator hierarchy that already exists, rather than being bolted onto a flat one from scratch.

नुक़सान: कोड पहले से ज़्यादा जटिल हो सकता है, क्योंकि पैटर्न लगाने का मतलब आमतौर पर नई सबक्लासेज़ का एक समूह जोड़ना है। यह सबसे आसानी से तब उतरता है जब इसे किसी पहले से मौजूद creator पदानुक्रम (hierarchy) में लगाया जा रहा हो, बजाय इसके कि इसे शुरू से एक सपाट (flat) संरचना पर जोड़ा जाए।

Relations with Other Patterns

Many designs start out with Factory Method — comparatively simple, and customizable purely through subclassing — and grow toward **Abstract Factory**, **Prototype**, or **Builder** as flexibility needs increase, each of those being more powerful but also more involved to set up.

कई डिज़ाइन्स Factory Method से शुरू होते हैं — तुलनात्मक रूप से सरल, और सिर्फ़ सबक्लासिंग के ज़रिए कस्टमाइज़ेबल — और जैसे-जैसे लचीलेपन की ज़रूरत बढ़ती है, ये **Abstract Factory**, **Prototype**, या **Builder** की ओर बढ़ते हैं, इनमें से हर एक ज़्यादा शक्तिशाली है लेकिन सेट करना भी ज़्यादा जटिल है।

An **Abstract Factory** class is frequently built out of a set of Factory Methods internally, though **Prototype** can also be used to compose those methods.

एक **Abstract Factory** क्लास अक्सर अंदर से Factory Methods के एक समूह से बनाई जाती है, हालाँकि उन मेथड्स को कंपोज़ करने के लिए **Prototype** का भी इस्तेमाल किया जा सकता है।

Factory Method pairs naturally with **Iterator**, letting a collection subclass return whichever iterator type is actually compatible with that collection.

Factory Method स्वाभाविक रूप से **Iterator** के साथ जोड़ी बनाता है, जिससे एक collection सबक्लास वही iterator टाइप लौटा सके जो उस collection के साथ असल में संगत (compatible) है।

Prototype, being inheritance-free, sidesteps inheritance's specific drawbacks, but pays for it with a more elaborate cloned-object initialization step. Factory Method leans on inheritance and needs no such initialization step in exchange.

Prototype, इनहेरिटेन्स-मुक्त होने के कारण, इनहेरिटेन्स की खास कमियों से बच जाता है, लेकिन इसके बदले क्लोन किए गए ऑब्जेक्ट के एक ज़्यादा विस्तृत इनिशियलाइज़ेशन चरण की कीमत चुकाता है। Factory Method इनहेरिटेन्स पर टिका है और बदले में ऐसे किसी इनिशियलाइज़ेशन चरण की ज़रूरत नहीं रखता।

Factory Method is, in a sense, a specialization of **Template Method** — and a single Factory Method call can itself serve as one step inside a larger Template Method sequence.

Factory Method, एक तरह से, **Template Method** का एक विशेषीकरण (specialization) है — और एक अकेली Factory Method कॉल खुद एक बड़े Template Method क्रम के अंदर एक चरण के तौर पर काम कर सकती है।

Code Examples (PHP)

The reference site links out to per-language example pages (C#, C++, Go, Java, PHP, Python, Ruby, Rust, Swift, TypeScript) with a dedicated, separately-URLed PHP example. What follows is original PHP written for this study, covering both scenarios discussed above: the logistics `Transport` example from the Problem/Solution sections, and the cross-platform `Dialog / Button` example from the Pseudocode section.

रेफ़रेंस साइट हर भाषा (C#, C++, Go, Java, PHP, Python, Ruby, Rust, Swift, TypeScript) के लिए अलग उदाहरण पेजेज़ की ओर लिंक करती है, जिसमें एक समर्पित, अलग-URL वाला PHP उदाहरण भी शामिल है। नीचे इसी अध्ययन के लिए लिखा गया ओरिजिनल PHP है, जो ऊपर बताए गए दोनों परिदृश्यों को कवर करता है: Problem/Solution सेक्शन्स का लॉजिस्टिक्स `Transport` उदाहरण, और Pseudocode सेक्शन का क्रॉस-प्लेटफ़ॉर्म `Dialog / Button` उदाहरण।

Example 1 — Logistics: Transport, Truck, Ship

```

<?php

interface Transport
{
    public function deliver(): string;
}

class Truck implements Transport
{
    public function deliver(): string
    {
        return "Delivering cargo by land, in a box on wheels.";
    }
}

class Ship implements Transport
{
    public function deliver(): string
    {
        return "Delivering cargo by sea, in a container on a hull.";
    }
}

abstract class Logistics
{
    // The factory method – every concrete Logistics subclass overrides this.
    abstract public function createTransport(): Transport;

    // Core business logic, written entirely against the Transport interface.
    public function planDelivery(): string
    {
        $transport = $this->createTransport();
        return "Planning delivery. " . $transport->deliver();
    }
}

class RoadLogistics extends Logistics
{
    public function createTransport(): Transport
    {
        return new Truck();
    }
}

class SeaLogistics extends Logistics
{
    public function createTransport(): Transport
    {
        return new Ship();
    }
}

// Client code – never references Truck or Ship directly.
foreach ([new RoadLogistics(), new SeaLogistics()] as $logistics) {
    echo $logistics->planDelivery() . "\n";
}

```

Expected output / अपेक्षित आउटपुट:

Planning delivery. Delivering cargo by land, in a box on wheels.
Planning delivery. Delivering cargo by sea, in a container on a hull.

Example 2 — Cross-Platform Dialog: Dialog, Button

```

<?php

interface Button
{
    public function render(): string;
}

class WindowsButton implements Button
{
    public function render(): string
    {
        return "Rendered a square, Windows-styled button.";
    }
}

class HtmlButton implements Button
{
    public function render(): string
    {
        return "Rendered an HTML <button> element.";
    }
}

abstract class Dialog
{
    // The factory method.
    abstract public function createButton(): Button;

    // Shared rendering logic – knows nothing about WHICH button it's using.
    public function render(): string
    {
        $button = $this->createButton();
        return "Rendering dialog window.\n" . $button->render();
    }
}

class WindowsDialog extends Dialog
{
    public function createButton(): Button
    {
        return new WindowsButton();
    }
}

class WebDialog extends Dialog
{
    public function createButton(): Button
    {
        return new HtmlButton();
    }
}

// Client picks the concrete Dialog subclass based on configuration –
// everything downstream of that one decision stays platform-agnostic.
function buildDialogForOs(string $os): Dialog
{
    return match ($os) {
        'windows' => new WindowsDialog(),
    };
}

```

```
        'web' => new WebDialog(),
        default => throw new \RuntimeException("Unknown operating system: {$os}"),
    };
}

$dialog = buildDialogForOs('web');
echo $dialog->render() . "\n";
```

Expected output / अपेक्षित आउटपुट:

```
Rendering dialog window.
Rendered an HTML <button> element.
```

Technical Words Glossary / तकनीकी शब्दों की शब्दावली

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Virtual Constructor	वर्चुअल कंस्ट्रक्टर	Factory Method को कभी-कभी वर्चुअल कंस्ट्रक्टर भी कहा जाता है।
Superclass	सुपरक्लास	Logistics सुपरक्लास factory मेथड डिक्लेयर करती है, इसे इम्प्लीमेंट नहीं करती।
Concrete Product	कॉन्क्रीट प्रोडक्ट	Truck और Ship , Transport इंटरफ़ेस के कॉन्क्रीट प्रोडक्ट्स हैं।
Decoupled	अलग-अलग किया हुआ, डीकपल्ड	factory मेथड, बिज़नेस लॉजिक को कॉन्क्रीट क्लासेज़ से डीकपल्ड रखता है।
Return Type	रिटर्न टाइप	factory मेथड का रिटर्न टाइप हमेशा साझा इंटरफ़ेस होना चाहिए, किसी एक कॉन्क्रीट क्लास का नहीं।
Object Pool	ऑब्जेक्ट पूल	महँगे ऑब्जेक्ट्स को दोबारा इस्तेमाल करने के लिए factory मेथड एक ऑब्जेक्ट पूल से इंस्टेंस लौटा सकता है।
Client Code	क्लाइंट कोड	क्लाइंट कोड सिर्फ़ Transport इंटरफ़ेस पर भरोसा करता है, Truck या Ship पर नहीं।
Single Responsibility Principle	सिंगल रिस्पॉन्सिबिलिटी प्रिंसिपल	सारा प्रोडक्ट-निर्माण कोड एक जगह रखना, इसी सिद्धांत का पालन है।
Open/Closed Principle	ओपन/क्लोज़्ड प्रिंसिपल	नया SeaLogistics जोड़ना, मौजूदा क्लाइंट कोड बदले बिना संभव है।
Creator Hierarchy	क्रिएटर पदानुक्रम	Logistics , RoadLogistics , और SeaLogistics मिलकर एक क्रिएटर पदानुक्रम बनाते हैं।

General Words Glossary / सामान्य शब्दों की शब्दावली

English Word	Hindi Meaning / हिंदी अर्थ	Example / उदाहरण
Disruptive	उथल-पुथल भरा, विघटनकारी	"The new policy caused disruptive changes across every department." नई नीति ने हर विभाग में उथल-पुथल भरे बदलाव किए।
Sprawling	फैला हुआ, बिखरा हुआ	"The city's sprawling suburbs stretched for miles." शहर के फैले हुए उपनगर मीलों तक फैले थे।
Elaborate (adjective)	विस्तृत, जटिल	"They planned an elaborate ceremony for the wedding." उन्होंने शादी के लिए एक विस्तृत समारोह की योजना बनाई।
Bolted onto	जोड़ दिया गया, चिपका दिया गया	"The extra room felt bolted onto the original house." अतिरिक्त कमरा मूल घर से चिपकाया हुआ-सा लगता था।
Genuinely	सचमुच, वास्तव में	"She was genuinely surprised by the gift." वह उपहार से सचमुच हैरान थी।
Structurally	संरचनात्मक रूप से	"The bridge was structurally unsound after the flood." बाढ़ के बाद पुल संरचनात्मक रूप से कमजोर हो गया था।
Payoff	फ़ायदा, नतीजा	"Years of saving finally had a payoff." सालों की बचत का आखिरकार फ़ायदा मिला।
Edges toward	धीरे-धीरे करीब जाना	"The conversation edged toward an uncomfortable topic." बातचीत धीरे-धीरे एक असहज विषय की ओर बढ़ी।
Sensible	उचित, समझदारी भरा	"It was a sensible decision given the circumstances." हालात को देखते हुए यह एक उचित फैसला था।
Bulk (noun, "the bulk of")	अधिकांश, बड़ा हिस्सा	"The bulk of the work was finished before the deadline." काम का बड़ा हिस्सा डेडलाइन से पहले पूरा हो गया था।

This document follows the section order of [refactoring.guru's Factory Method page](#) (commercially licensed content tied to their eBook and course) but is written independently — original English/Hindi explanations and original PHP code — rather than reproducing their material.

यह दस्तावेज़ [refactoring.guru](#) के *Factory Method* पेज (उनके eBook और कोर्स से जुड़ी व्यावसायिक रूप से लाइसेंस प्राप्त सामग्री) के सेक्शन-क्रम को फ़ॉलो करता है, लेकिन स्वतंत्र रूप से लिखा गया है — ओरिजिनल अंग्रेज़ी/हिंदी व्याख्याएँ और ओरिजिनल PHP कोड — बजाय इसके कि उनकी सामग्री को दोबारा प्रस्तुत किया जाए।